Prakash Thapa
D20A -32

Experiment No.: 4

AIM
To develop a blockchain-powered web application using Solidity programming language on Remix IDE and integrate a MetaMask wallet to deploy and perform transactions with a smart contract on the Sepolia Testnet.

COURSE OUTCOMES (CO)
- Understand the basic working of Ethereum-based blockchain applications.
- Learn to create and compile smart contracts using Solidity.
- Understand the role of MetaMask as a blockchain wallet.
- Deploy and interact with a smart contract using Remix IDE.
- Perform blockchain transactions on a test network.

SOFTWARE / TOOLS REQUIRED
- MetaMask Wallet
- Remix IDE
- Web Browser (Brave/Chrome)
- Sepolia Ethereum Testnet
- Internet Connection
- Sepolia Blockchain Explorer

THEORY

1. Blockchain
Blockchain is a distributed and decentralized ledger used to store transactions and data in a secure and tamper-resistant manner. Transactions are grouped into blocks and linked together using cryptographic techniques.
Ethereum is a blockchain platform that supports programmable applications through smart contracts.

2. Smart Contract
A smart contract is a program stored and executed on a blockchain. It contains predefined rules and functions that can be executed when the required conditions are satisfied.
Smart contracts on Ethereum-compatible networks are commonly written using Solidity.

3. Solidity
Solidity is a high-level, statically typed programming language designed for writing smart contracts for Ethereum and Ethereum-compatible blockchains.

A Solidity contract can contain:

- Variables
- Functions
- Constructors
- Events
- Modifiers
- Data structures

4. MetaMask

MetaMask is a cryptocurrency wallet that acts as an interface between a web application and blockchain networks.
In this experiment, MetaMask is used to:

- Create/import a blockchain account.
- Connect to the Sepolia Testnet.
- Hold test ETH.
- Approve blockchain transactions.
- Sign transactions using the wallet account.

5. Sepolia Testnet

Sepolia is an Ethereum test network used for development and testing without using real ETH. The ETH obtained from a Sepolia faucet has no real monetary value.
The testnet allows developers to deploy and interact with smart contracts before deploying them on the Ethereum mainnet.

6. Remix IDE

Remix IDE is a browser-based development environment for writing, compiling, deploying and interacting with Solidity smart contracts.
For this experiment, Remix is connected to MetaMask through the Injected Provider environment.

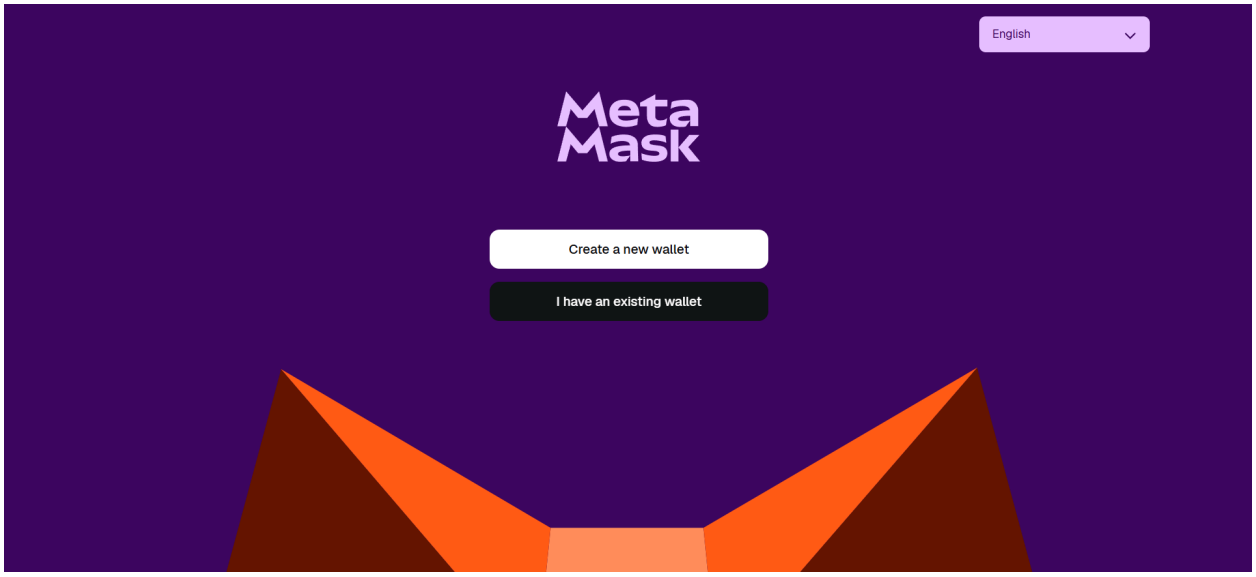
PROCEDURE

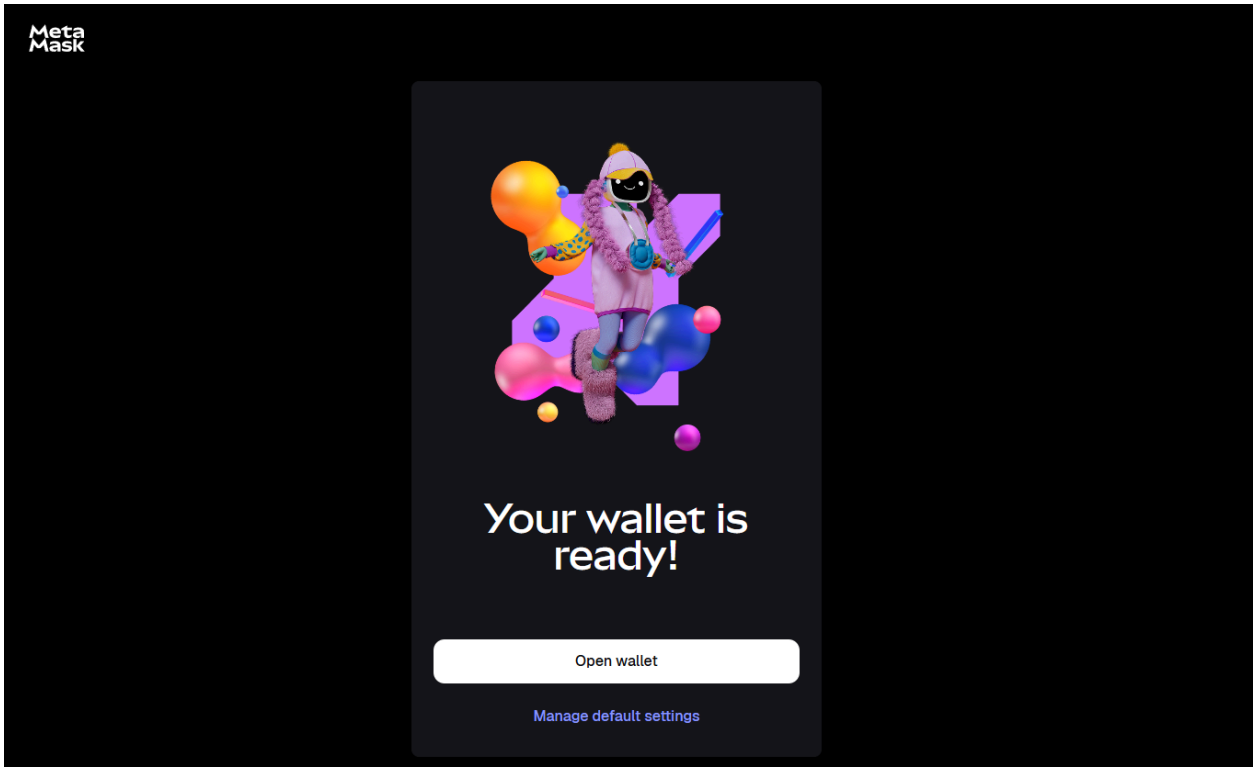
Step 1: Set Up MetaMask

1. Open the MetaMask extension in the web browser.
2. Create a new wallet or import an existing wallet.
3. Complete the wallet setup.
4. Open the network selection menu.
5. Select Sepolia Test Network.
6. Copy the wallet address for obtaining test ETH.

Output:

MetaMask wallet was successfully created and configured for the Sepolia Testnet.



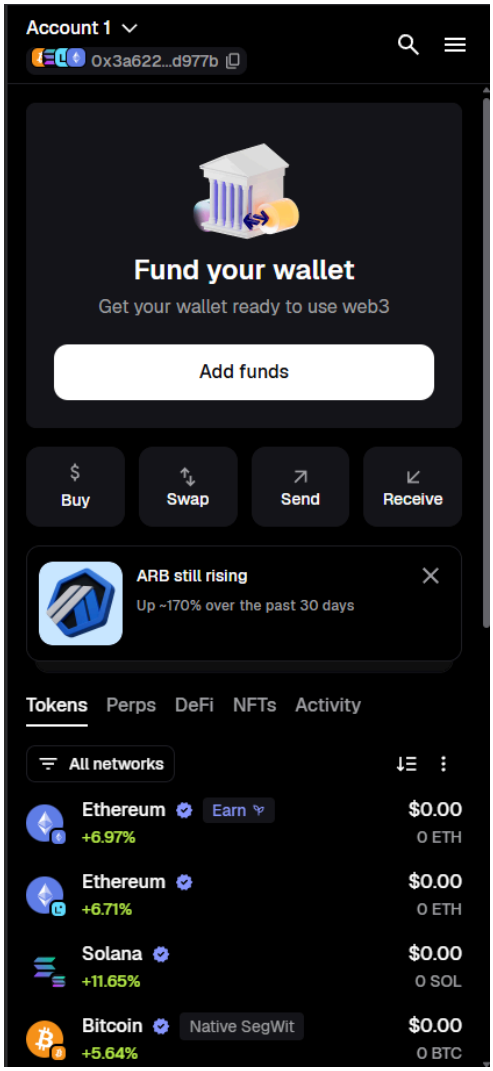


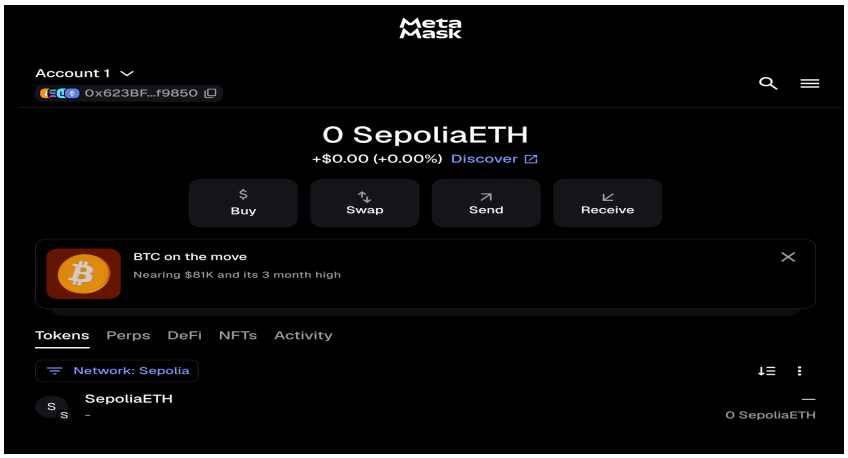
Step 2: Fund the Wallet

1. Open a Sepolia ETH faucet.
2. Enter the MetaMask wallet address.
3. Request test ETH.
4. Wait for the transaction to be processed.
5. Open MetaMask and verify that the Sepolia ETH has been received.

Output:

The MetaMask wallet was funded with Sepolia test ETH for performing blockchain transactions.





Step 3: Open Remix IDE

1. Open Remix IDE in the browser.
2. Create a new Solidity file.
3. Give the file an appropriate name.

4. Write the Solidity smart contract based on the selected mini-project.

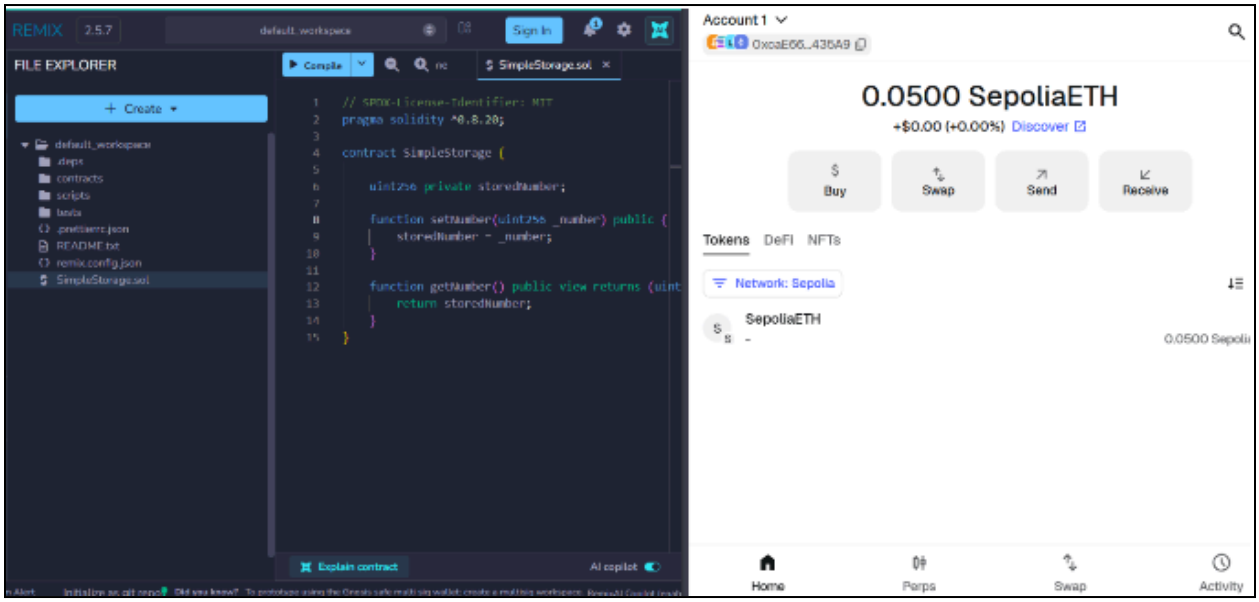
A simple example contract used for demonstrating contract deployment and interaction is:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Exp4 {
   string public message;

   constructor(string memory _message) {
      message = _message;
   }

   function setMessage(string memory _message) public {
      message = _message;
   }
}
```

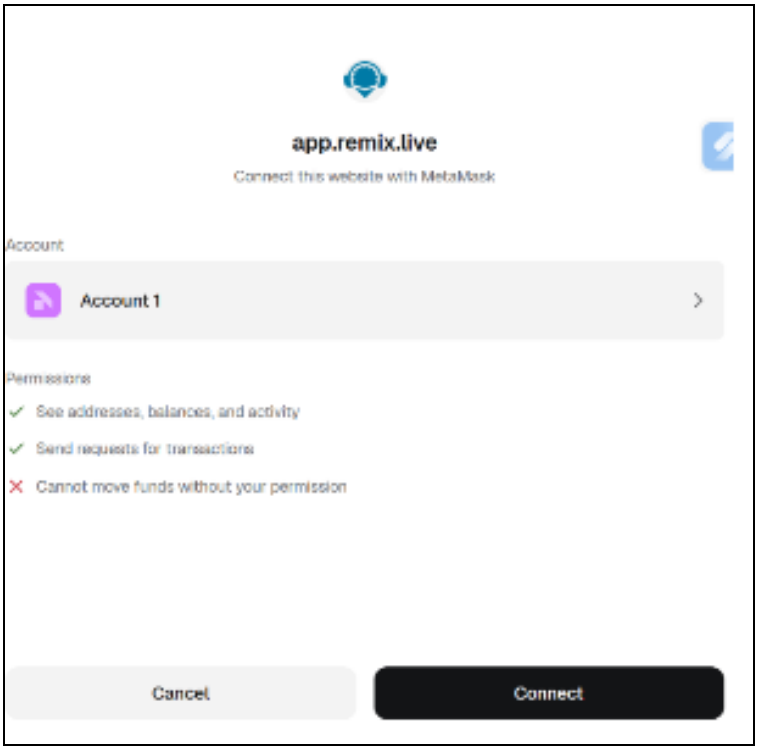


Step 4: Compile the Smart Contract

1. Open the Solidity Compiler tab in Remix.
2. Select a compiler version compatible with the pragma statement.
3. Click Compile Exp4.sol.
4. Verify that there are no compilation errors.

Output:

The Solidity smart contract was successfully compiled.



Step 5: Connect Remix with MetaMask

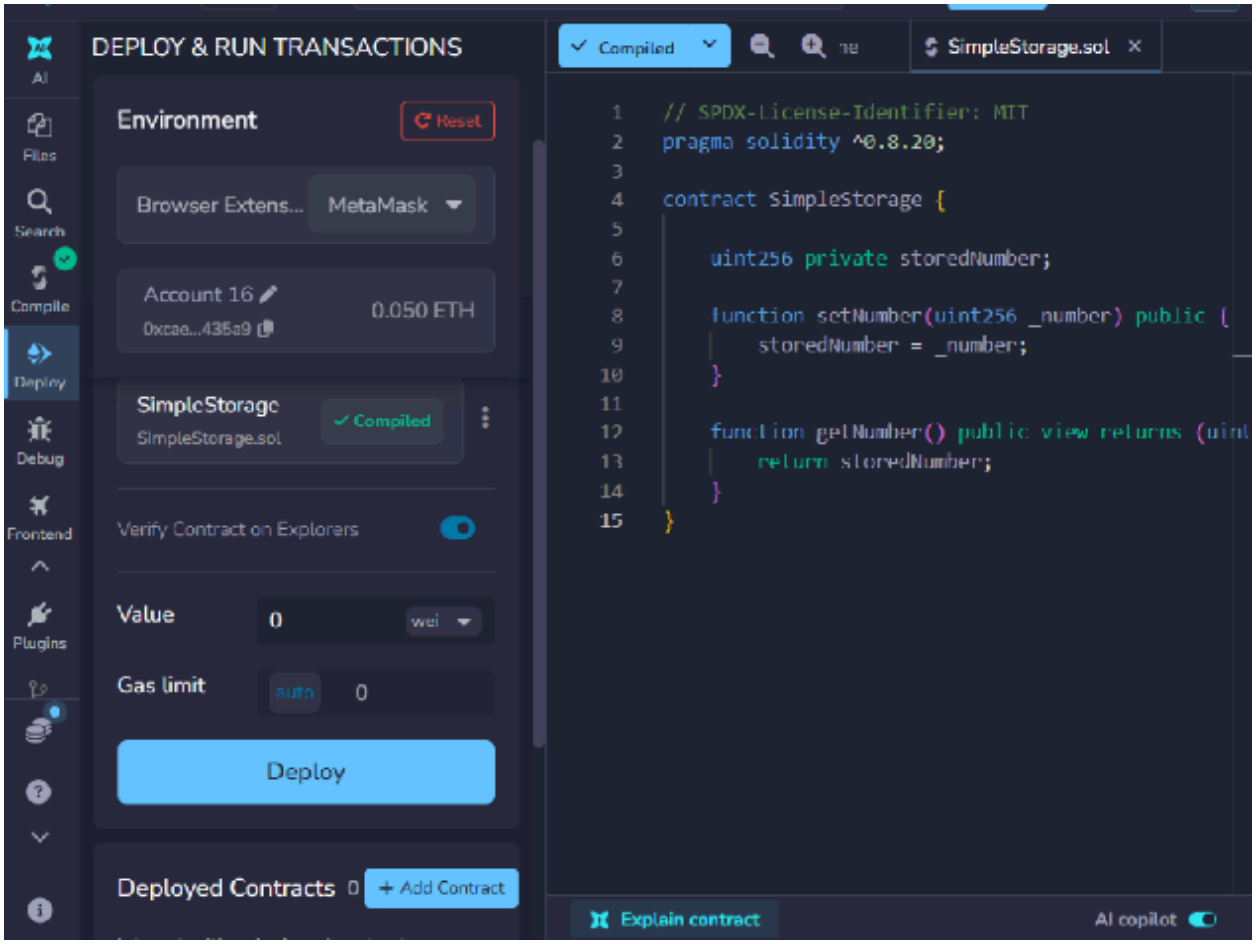
1. Open the Deploy & Run Transactions tab in Remix.
2. Under Environment, select:

Injected Provider - MetaMask

3. MetaMask opens and asks for permission to connect.
4. Approve the connection.
5. Verify that the account displayed in Remix corresponds to the MetaMask account.
6. Verify that the selected network is Sepolia Testnet.

Output:

Remix IDE was successfully connected to the MetaMask wallet through the Sepolia Testnet.



Step 6: Deploy the Smart Contract

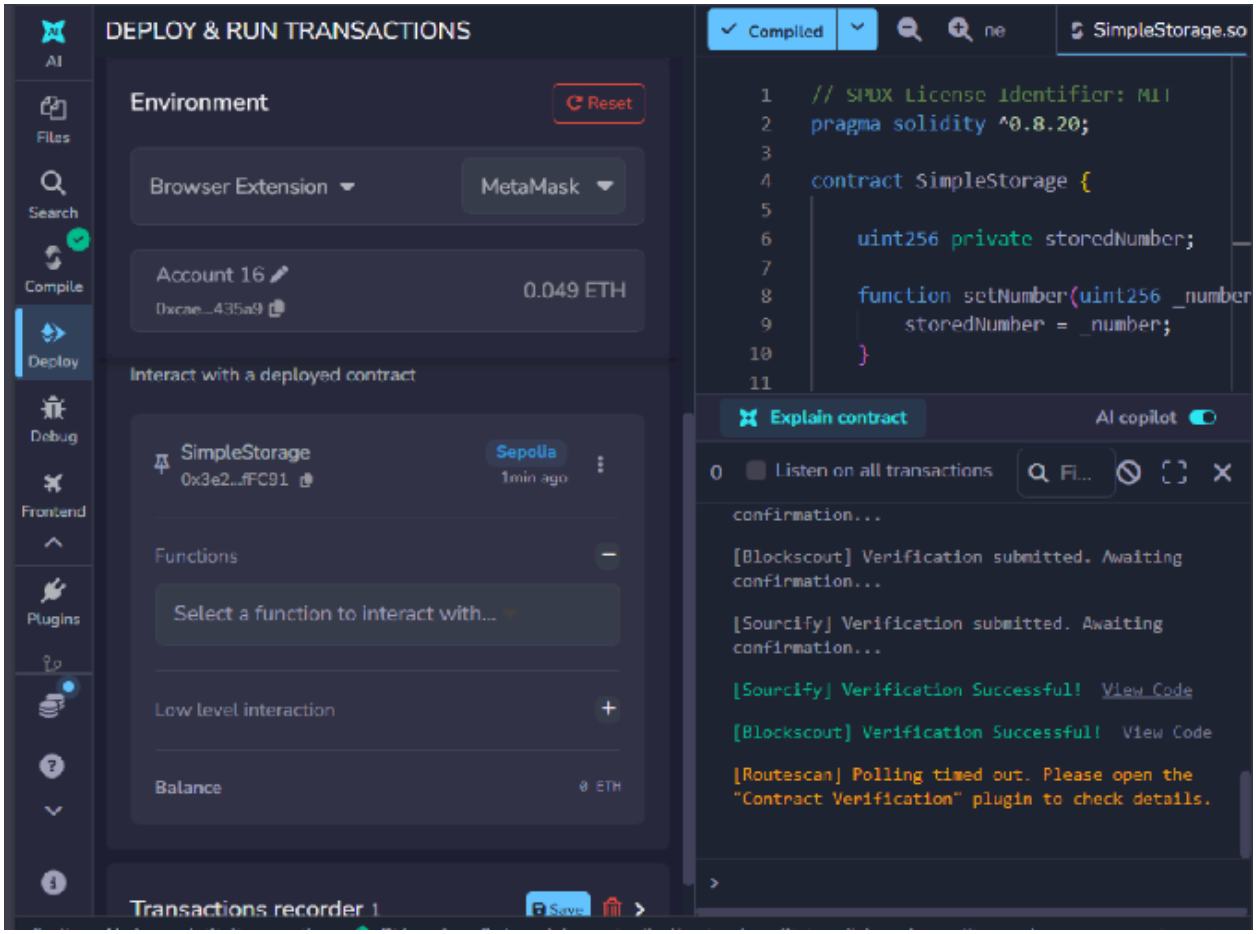
1. In the Deploy & Run Transactions tab, select the compiled contract.
2. Enter the required constructor value, if applicable.

For the example contract:
"Hello from Prakash - BDA/Blockchain Lab Exp 4"

3. Click Deploy.
4. MetaMask displays the transaction confirmation.
5. Review the transaction details and gas fee.
6. Confirm the transaction in MetaMask.
7. Wait for the transaction to be included in the Sepolia blockchain.
8. After successful deployment, the contract appears under Deployed Contracts in Remix.

Output:

The smart contract was successfully deployed on the Sepolia Testnet.



Step 7: Interact with the Smart Contract

After deployment, Remix displays the available public functions.

For the example contract:

message()

- Used to read the currently stored message.
- It does not require a blockchain transaction because it only reads blockchain data.

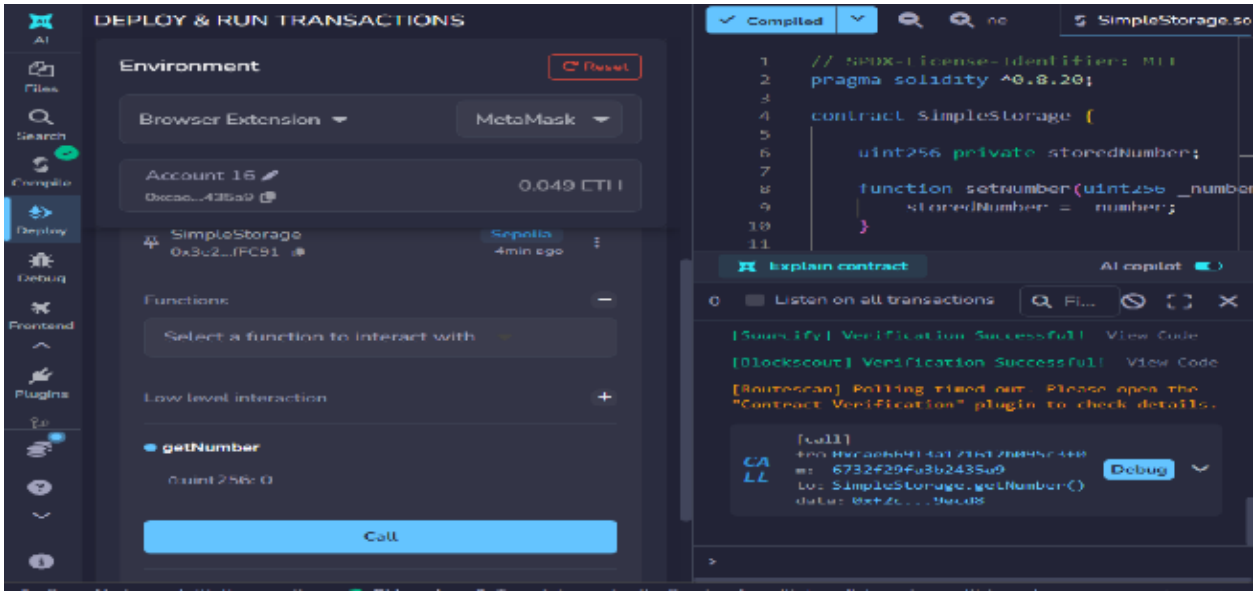
setMessage()

- Used to change the stored message.
- It requires a blockchain transaction because it modifies blockchain state.

To update the message:

1. Enter the new message in the setMessage field.
2. Click transact.
3. MetaMask opens the transaction confirmation window.
4. Review the transaction.
5. Click Confirm.
6. Wait for transaction confirmation.
7. Click message() to verify the updated value.

Output:
The smart contract was successfully interacted with through Remix, and the stored blockchain data was updated through a MetaMask-approved transaction.



Step 8: Verify the Transaction on Sepolia Explorer

1. Copy the transaction hash displayed after performing the transaction.
2. Open the Sepolia blockchain explorer.
3. Search for the transaction hash.
4. Verify the transaction details.

The explorer can show information such as:

- Transaction hash
- From address
- To address / contract address
- Block number
- Transaction status
- Gas used
- Timestamp

Output:
The smart contract transaction was successfully verified on the Sepolia blockchain explorer.

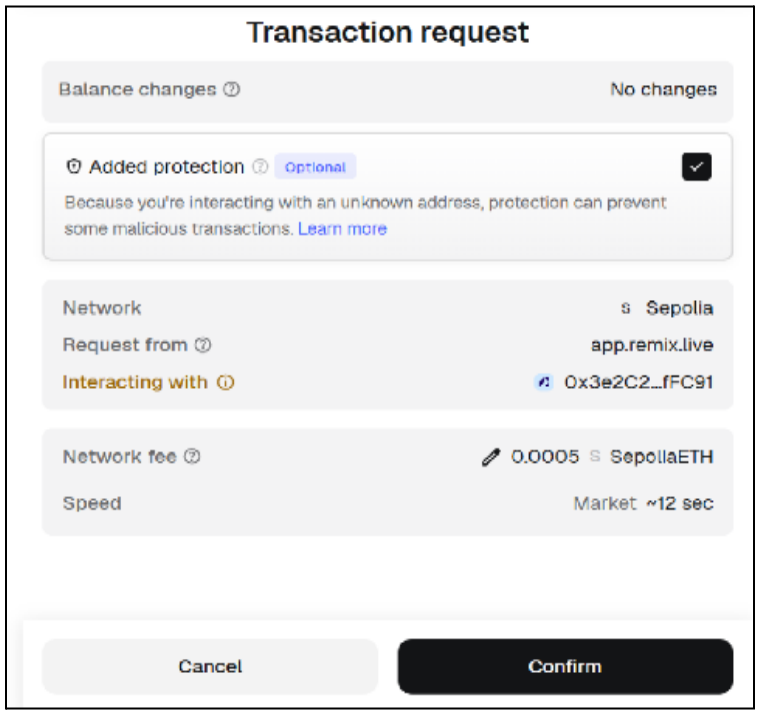
TRANSACTION FLOW

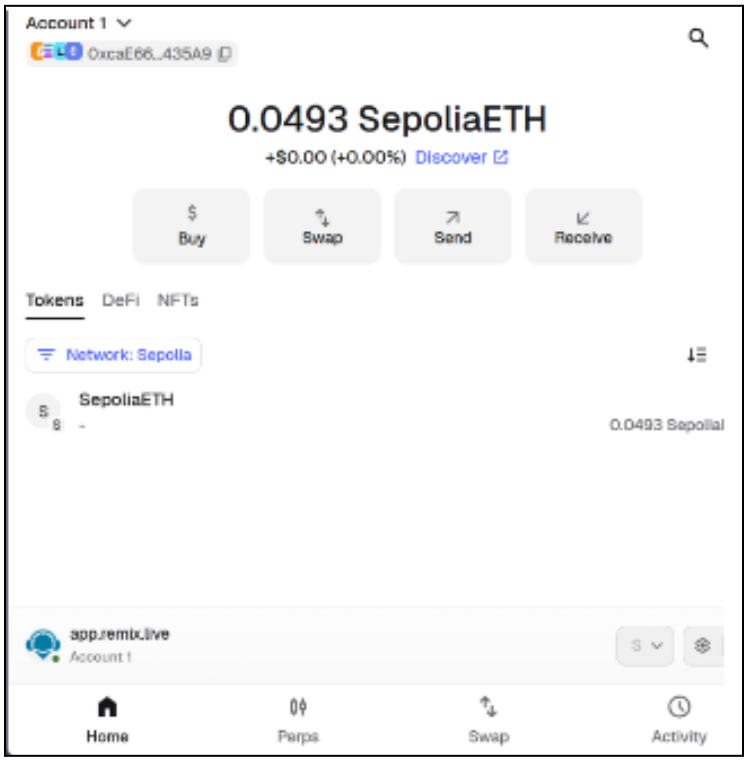
The overall working of the experiment can be represented as:

User → Remix IDE → MetaMask → Sepolia Testnet → Smart Contract

1. The user interacts with the smart contract through Remix.
2. Remix sends the transaction request to MetaMask.
3. MetaMask asks the user to approve and sign the transaction.
4. The transaction is submitted to the Sepolia network.

5. Validators process the transaction.
6. The smart contract executes the requested function.
7. The transaction is recorded on the blockchain.
8. The result can be verified using Remix and the Sepolia Explorer.





RESULT

The Solidity smart contract was successfully created and compiled using Remix IDE. MetaMask was configured and connected to the Sepolia Testnet. The smart contract was deployed using MetaMask, and transactions were successfully performed and verified on the Sepolia blockchain explorer.

CONCLUSION

This experiment demonstrated the development and deployment of a blockchain-based application using Solidity, Remix IDE, MetaMask, and the Sepolia Ethereum Testnet. It provided practical understanding of smart-contract deployment, wallet-based transaction signing, blockchain interaction, and transaction verification using a blockchain explorer.